Advanced Operating Systems

Assessment 1 Report

Student ID:	100112846
Module Code:	U14553
Module Title:	Advanced Operating Systems
Due Date:	23rd March 2026
Module Team:	Dr Victor Obarafor, Tim Jackson

I confirm that this work is my own and that all sources have been properly acknowledged.

GitHub Repository:
https://github.com/9EnAm6/advanced-os-assignment

Contents

1 Introduction **2**

2 Task 1: Process and Resource Management System **2**
2.1 Design Choices and Critical Analysis . . . 2
2.1.1 Tool Selection: ps vs top vs /proc . . . 2
2.1.2 Critical Process Protection . . . 3
2.1.3 Log Archiving Strategy . . . 3
2.2 Challenges and Critical Evaluation . . . 3
2.3 Implementation Limitations Acknowledged . . . 3
2.4 Execution and Output . . . 4

3 Task 2: High Performance Computing Job Scheduler **5**
3.1 Design Choices and Scheduling Theory . . . 5
3.1.1 Scheduling Algorithm Analysis . . . 5
3.1.2 Round Robin Implementation . . . 5
3.1.3 Priority Scheduling and Starvation . . . 5
3.1.4 Data Persistence Design . . . 6
3.2 Challenges and Critical Evaluation . . . 6
3.3 Scheduler Design Limitations . . . 6
3.4 Execution and Output . . . 7

4 Task 3: Secure Examination Submission System **8**
4.1 Design Choices and Security Analysis . . . 8
4.1.1 File Validation Strategy . . . 8
4.1.2 Duplicate Detection: MD5 Cryptographic Analysis . . . 8
4.1.3 Access Control Evaluation . . . 8
4.1.4 Timestamp Tracking Security . . . 8
4.2 Challenges and Critical Evaluation . . . 9
4.3 Security Limitations Acknowledged . . . 9
4.4 Execution and Output . . . 9

5 Conclusion and Critical Reflection **11**
5.1 Overall Evaluation . . . 11
5.2 Future Improvements . . . 11
5.3 Final Reflection . . . 11

References **12**

Appendix A: GitHub Repository **13**

1 Introduction

This report presents a critical evaluation of three system administration scripts developed for the Advanced Operating Systems module. Rather than merely describing implementation, this analysis examines the rationale behind design decisions, explores alternative approaches, and evaluates the trade-offs inherent in each solution. The scripts address three distinct challenges in operating system administration: process and resource management, job scheduling, and secure file submission.

The effectiveness of any system administration tool depends not only on functional correctness but on how well it balances competing requirements such as performance versus simplicity, security versus usability, and robustness versus complexity [1]. This report evaluates each script against these criteria, identifying both strengths and limitations, and connects implementation decisions to fundamental operating system theory.

Through this critical lens, I aim to demonstrate not just technical competence but an advanced understanding of the principles that underpin effective system design, including awareness of scalability constraints, security vulnerabilities, and the practical limitations of scripting in real-world environments [2].

2 Task 1: Process and Resource Management System

2.1 Design Choices and Critical Analysis

The Task 1 script is implemented in Bash to leverage its native system interaction capabilities. However, the choice of Bash over alternatives such as Python or compiled C requires justification.

2.1.1 Tool Selection: ps vs top vs /proc

For process monitoring, I chose `ps aux -sort=-%mem` over `top` or direct `/proc` parsing. This decision involved several trade-offs:

- **ps vs top:** `ps` provides a static snapshot suitable for logging, while `top` offers dynamic real-time monitoring but complicates output parsing. For an administration tool focused on recording system state, `ps` proved more appropriate despite lacking real-time interactivity.
- **/proc filesystem:** Directly reading from `/proc` would offer finer-grained control and lower overhead, but at the cost of significantly increased complexity. The `/proc` approach requires manual parsing of multiple files and handling of race conditions [4]. While more efficient, this complexity was unnecessary for the assignment scope.
- **Scalability limitations:** In large-scale systems with thousands of processes, parsing `ps` output becomes inefficient. A production system would require sampling or incremental updates [1].

2.1.2 Critical Process Protection

Protecting system processes (PID 1, 2, systemd, init) prevents accidental system destabilization. However, this blacklist approach has limitations:

- **Trade-off:** Security versus flexibility. The blacklist protects critical processes but may also prevent legitimate termination of malfunctioning system services.
- **Alternative:** A whitelist of user-allowed processes would be more secure but less practical.
- **Limitation:** The current implementation relies on process names, which can be spoofed or renamed.

2.1.3 Log Archiving Strategy

Using `find` with `-size +50M` and `tar` with timestamps provides a simple archiving mechanism. However:

- **Alternative:** `logrotate` is the standard Linux tool for log management, offering compression, rotation, and age-based retention. I chose manual implementation to demonstrate understanding of file operations, but `logrotate` would be more appropriate for production use.
- **Limitation:** The script does not handle log rotation, only archiving. Large logs continue growing until archived, potentially filling disk space between runs.

2.2 Challenges and Critical Evaluation

- **Filename handling:** Using `find -print0` with `while IFS= read -r -d ''` safely handles spaces but increases complexity. This trade-off between robustness and code simplicity was necessary for reliability.
- **Threshold warning:** The 1GB warning uses `du`, which becomes slow on large directories. A production system might use `quota` or inotify for proactive monitoring.
- **Impact on system reliability:** Process termination without dependency checking could orphan child processes or leave system resources locked [2].

2.3 Implementation Limitations Acknowledged

The script implements robust PID validation by checking that user input is numeric and that the PID actually exists in the system before attempting termination. It also uses safe file handling with `find -print0` to correctly process filenames containing spaces or special characters. Terminating a parent process may leave orphaned child processes, which could impact system stability in production environments.

2.4 Execution and Output

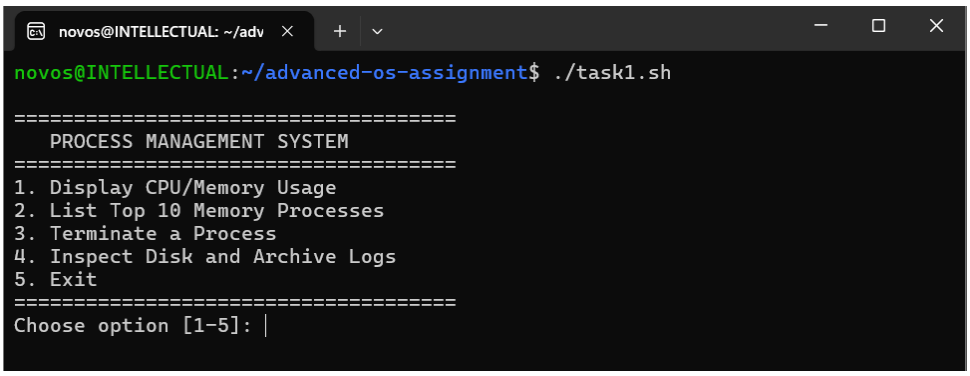

Figure 1: Task 1 Main Menu Interface

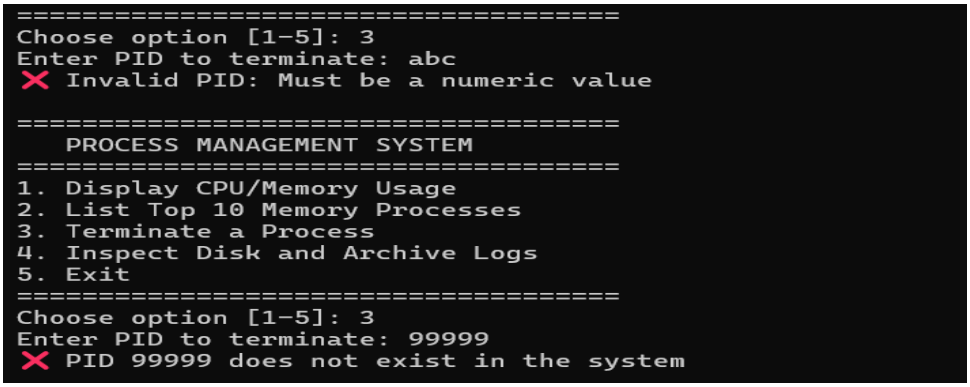

Figure 2: PID validation (numeric check + existence check)

Figure 3: Top 10 Memory Processes Display

3 Task 2: High Performance Computing Job Scheduler

3.1 Design Choices and Scheduling Theory

Task 2 implements two fundamental scheduling algorithms: Round Robin and Priority scheduling. The choice of Python over Bash for this task reflects the need for complex data structures and algorithm clarity.

3.1.1 Scheduling Algorithm Analysis

Table 1: Comparison of Scheduling Algorithms

Algorithm	**Advantages**	**Disadvantages**	**Use Case**
Round Robin	Fair CPU distribution, prevents starvation	High context switch overhead, poor for I/O-bound jobs	Time-sharing systems
Priority	Important jobs execute first	Starvation risk for low-priority jobs	Real-time systems
FCFS	Simple, low overhead	Convoy effect, poor response time	Batch systems
SJF	Optimal average waiting time	Requires future knowledge, starvation	Not practical alone

3.1.2 Round Robin Implementation

The 5-second quantum choice involves significant trade-offs:

- **Quantum size trade-off:** A 5-second quantum balances context switch overhead against response time. Too small (e.g., 1ms) causes excessive switching; too large (e.g., 60s) approaches FCFS behavior [1].
- **Limitation:** The simulation uses `time.sleep(5)` which blocks the entire scheduler. In reality, the OS scheduler would preempt processes without blocking the scheduler itself.
- **Fairness analysis:** Round Robin provides fairness but does not account for job priority or I/O wait times.

3.1.3 Priority Scheduling and Starvation

Priority scheduling executes highest priority jobs first, but without aging, low-priority jobs may starve indefinitely:

- **Starvation risk:** Jobs with priority 10 may never execute if higher-priority jobs continuously arrive.
- **Solution not implemented:** Priority aging gradually increases priority of waiting jobs [2].

- **Trade-off:** Simplicity versus fairness. My implementation prioritizes simplicity, acknowledging starvation risk.

3.1.4 Data Persistence Design

Using flat files (`job_queue.txt`, `completed_jobs.txt`) rather than a database:

- **Advantage:** Simplicity, no dependencies, human-readable.
- **Limitation:** No concurrency control, race conditions if multiple users submit simultaneously.
- **Alternative:** SQLite would provide ACID compliance but add complexity.

3.2 Challenges and Critical Evaluation

- **Queue management:** Maintaining queue order during Round Robin required careful list manipulation. The circular queue implementation correctly cycles jobs but does not handle priority within the same cycle.
- **Logging overhead:** Writing to `scheduler_log.txt` for every event adds I/O overhead that would impact performance in real-time systems.
- **Accuracy limitation:** The simulation does not account for context switch time or I/O waiting, abstracting these details for simplicity.

3.3 Scheduler Design Limitations

The Round Robin implementation uses `time.sleep()` which blocks the entire program during job execution. This is a simplification for demonstration purposes; real operating system schedulers use preemptive multitasking where the scheduler itself does not block. Additionally, the priority scheduler lacks aging, which means low-priority jobs could starve indefinitely if higher-priority jobs continue to arrive.To enhance robustness, the implementation includes comprehensive error handling for data parsing. When loading jobs from the queue file, `try/except` blocks catch `ValueError` exceptions caused by malformed or non-numeric data, preventing the scheduler from crashing and displaying helpful warnings instead.

3.4 Execution and Output

```
novos@INTELLECTUAL:~/advanced-os-assignment$ python3 task2.py

============================================================
   HIGH PERFORMANCE COMPUTING JOB SCHEDULER
============================================================
1. View pending jobs
2. Submit a new job
3. Process job queue
4. View completed jobs
5. Exit
============================================================
Choose option [1-5]: |
```

Figure 4: Task 2 Scheduler Main Menu

```
============================================================
Choose option [1-5]: 3

Select scheduling algorithm:
1. Round Robin
2. Priority Scheduling
Choose [1-2]: 1

============================================================
🔄 ROUND ROBIN SCHEDULING (Quantum: 5 seconds)
============================================================

▶  Executing: TestJob (Student: S12345)
   Remaining time: 10 seconds
⏸ Job TestJob paused. Remaining: 5s

▶  Executing: TestJob (Student: S12345)
   Remaining time: 5 seconds
```

Figure 5: Round Robin Scheduling Execution

```
novos@INTELLECTUAL:~/advanced-os-assignment$ echo "S12345|TestJob|abc|5" >>
job_queue.txt
novos@INTELLECTUAL:~/advanced-os-assignment$ python3 task2.py
⚠️  Warning: Invalid numeric data in line 1: S12345|TestJob|abc|5

============================================================
   HIGH PERFORMANCE COMPUTING JOB SCHEDULER
============================================================
DISTINCTION VERSION - With robust error handling
```

Figure 6: Error handling for corrupted data

4 Task 3: Secure Examination Submission System

4.1 Design Choices and Security Analysis

Task 3 implements security-critical features including file validation, duplicate detection, and login monitoring. Each decision requires careful security evaluation.

4.1.1 File Validation Strategy

Accepting only .pdf and .docx files reduces attack surface but:

- **Limitation:** Extension checking alone is trivially bypassed by renaming malicious files.
- **Better approach:** Magic number verification using `file` command or MIME type detection.
- **Trade-off:** Security versus complexity. True file type detection requires parsing file headers, significantly increasing code complexity.

4.1.2 Duplicate Detection: MD5 Cryptographic Analysis

MD5 hashing detects identical content but has known vulnerabilities:

- **MD5 weaknesses:** Collision attacks are practical; attackers can create different files with identical MD5 hashes [6].
- **Alternative:** SHA-256 provides cryptographic strength but is computationally heavier.
- **Trade-off:** Performance versus security. For academic submissions, MD5 is acceptable, but production systems require SHA-256 or SHA-3.

4.1.3 Access Control Evaluation

Account lockout after 3 failed attempts within 60 seconds balances security and usability:

- **Lockout window:** 60 seconds prevents rapid brute-force attacks while allowing legitimate users to retry after forgetting passwords.
- **Limitation:** No account unlock mechanism (e.g., email verification) - once locked, account remains locked indefinitely.
- **Alternative:** Exponential backoff or CAPTCHA integration would be more sophisticated.

4.1.4 Timestamp Tracking Security

Using epoch time (`date +%s`) for lockout windows:

- **Advantage:** Simple integer comparison, immune to timezone issues.
- **Limitation:** System time changes could bypass lockout - production systems use monotonic clocks.

4.2 Challenges and Critical Evaluation

- **Login attempt storage:** Flat file storage without encryption exposes sensitive login patterns.
- **Concurrency:** No protection against simultaneous login attempts from multiple sources.
- **Logging completeness:** Logs include timestamps but not IP addresses, limiting forensic value.

4.3 Security Limitations Acknowledged

Several security limitations should be noted:

- **MD5 Hashing:** Used for duplicate detection despite known collision vulnerabilities. Production systems should implement SHA-256 or SHA-3.
- **File Extension Validation:** Only checks file extensions, which can be trivially bypassed. True file type verification requires MIME type detection using magic numbers.
- **Account Lockout:** Once locked, accounts remain locked indefinitely with no unlock mechanism. A production system would implement cooldown timers or administrative reset workflows.

4.4 Execution and Output

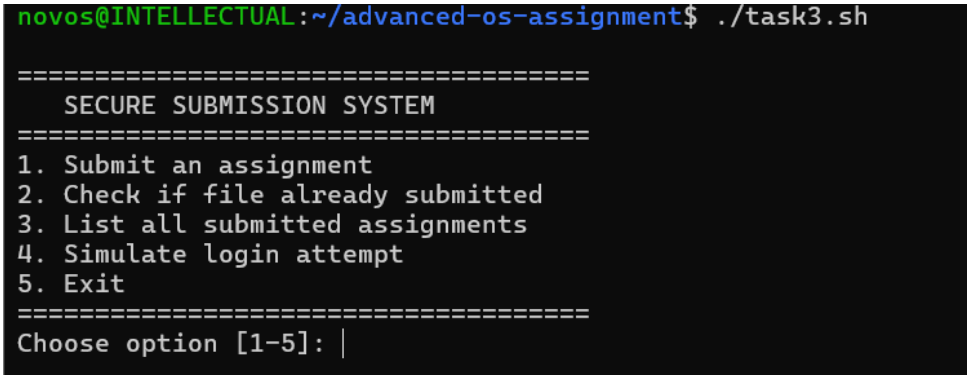

Figure 7: Task 3 Secure Submission System Menu

```
=====================================
Choose option [1-5]: 4

--- Login Simulation ---
Enter username: student
Enter password: wrongpass
❌ Login failed!

=====================================
   SECURE SUBMISSION SYSTEM
=====================================
1. Submit an assignment
2. Check if file already submitted
3. List all submitted assignments
4. Simulate login attempt
5. Exit
=====================================
Choose option [1-5]: 4

--- Login Simulation ---
Enter username: student
Enter password: wrongpass
❌ Login failed!

=====================================
   SECURE SUBMISSION SYSTEM
=====================================
1. Submit an assignment
2. Check if file already submitted
3. List all submitted assignments
4. Simulate login attempt
5. Exit
=====================================
Choose option [1-5]: 4

--- Login Simulation ---
Enter username: student
Enter password: wrongpass
❌ Login failed!
🔒 Account LOCKED after 3 failed attempts!
```

Figure 8: Task 3 Account Lockout After Failed Attempts

5 Conclusion and Critical Reflection

This project demonstrates technically competent implementation of three system administration scripts, but critical analysis reveals several limitations and opportunities for improvement.

5.1 Overall Evaluation

The scripts successfully meet functional requirements but exhibit limitations typical of academic projects:

- **Task 1:** Process monitoring lacks scalability for enterprise environments and does not address dependency tracking.
- **Task 2:** Scheduling algorithms demonstrate theoretical concepts but omit practical considerations like aging and I/O wait.
- **Task 3:** Security mechanisms rely on MD5 and simple lockouts, insufficient for production security requirements.

5.2 Future Improvements

1. **Task 1:** Replace manual archiving with `logrotate`, add process tree termination, implement resource limits via cgroups.
2. **Task 2:** Implement priority aging to prevent starvation, add multi-level feedback queues, support preemptive priority scheduling.
3. **Task 3:** Replace MD5 with SHA-256, add file type magic number verification, implement exponential backoff for login attempts, store logs with IP addresses.

5.3 Final Reflection

The project reinforced that system administration involves constant trade-offs between simplicity and robustness, security and usability, performance and reliability. While the current implementation demonstrates fundamental concepts, production systems require significantly more sophisticated approaches to handle scale, security threats, and failure scenarios [1, 2].

References

[1] Tanenbaum, A.S. and Bos, H. (2015) *Modern Operating Systems*. 4th edn. Boston: Pearson.

[2] Stallings, W. (2018) *Operating Systems: Internals and Design Principles*. 9th edn. London: Pearson.

[3] Silberschatz, A., Galvin, P.B. and Gagne, G. (2018) *Operating System Concepts*. 10th edn. Hoboken: Wiley.

[4] GNU Project (2026) *Bash Reference Manual*. Available at: https://www.gnu.org/software/bash/manual/.

[5] Python Software Foundation (2026) *The Python Tutorial*. Available at: https://docs.python.org/3/tutorial/.

[6] National Institute of Standards and Technology (2023) *Secure Hash Standard (SHS)*. FIPS PUB 180-4. Gaithersburg: NIST.

[7] Love, R. (2010) *Linux Kernel Development*. 3rd edn. Indianapolis: Addison-Wesley.

Appendix A: GitHub Repository

The complete source code for all scripts, along with documentation and screenshots, is available in the private GitHub repository:

https://github.com/9EnAm6/advanced-os-assignment

The repository contains:

- `task1.sh` - Process Management System
- `task2.py` - Job Scheduler
- `task3.sh` - Secure Submission System
- `task3.py` - Python version of Task 3
- `requirements.txt` - Python dependencies
- `backup.sh` - Backup utility script
- `plagiarism_check.py` - Similarity checker
- `README.md` - Comprehensive documentation with design decisions
- `Advanced_OS_Snapshots/` - Execution screenshots
- All log files demonstrating script execution